Analyzing the Effect of Network Width on Graph Transformers

What has been done, found and learned: 1 October – 10 November 2026

Experiment report, written from scratch

In one paragraph. We measure how **wide** a transformer must be to learn whether two nodes of a graph are connected, and how that width grows with the number of nodes n. With the standard input (each node's row of the adjacency matrix) the required width grows almost like n^2 ($n^{1.74}$) and no training trick removes the growth. Changing the **input encoding** changes the growth: handing the model 2-hop or 4-hop information lowers it to $n^{1.27}$ and $n^{1.03}$, and putting the edges directly into the attention scores (**edge bias**) makes the width almost independent of n: 4–7 numbers per node are enough for every size from 32 to 128 nodes ($n^{0.32}$), where the standard input needs 22 at $n = 32$ and 60 at $n = 56$. Depth is held fixed in this comparison; it is a setting, not the variable.

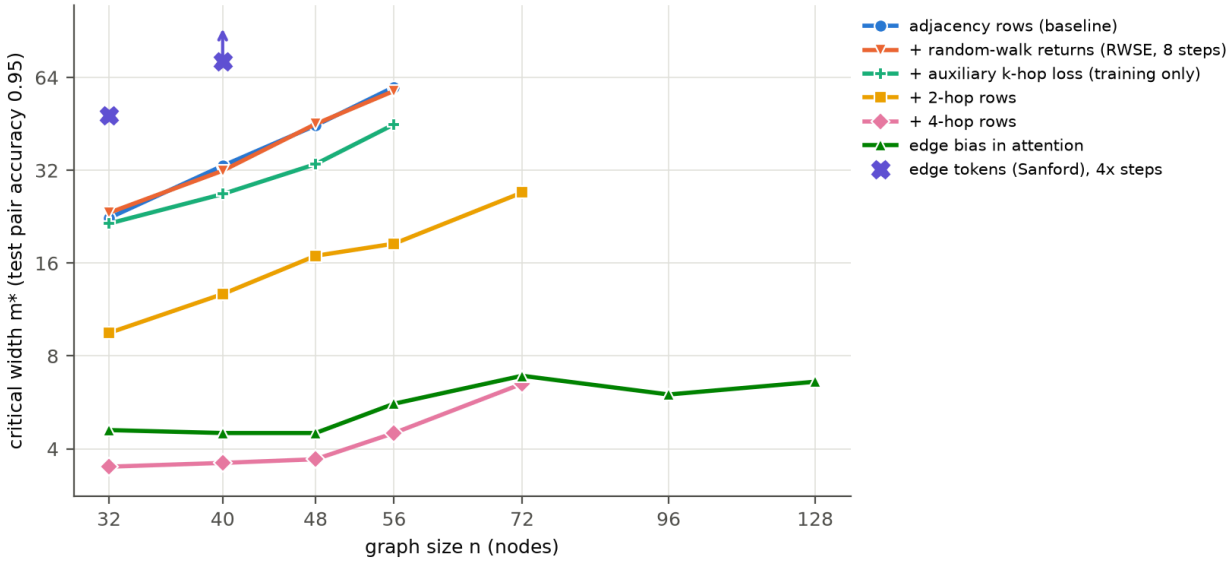


Figure 1: The main result. Critical width m^* (the smallest width at which a model gets 95 % of node pairs right on unseen graphs) against graph size, for every input encoding and method tried. Both axes are logarithmic, so a straight line is a power law $m^* \propto n^k$ and its slope is k. Edge bias is measured at depth 12, all other lines at depth 6 (section 10.6 explains why, and what is still missing to compare them at one depth).

Contents

1 Summary 3
1.1 The question 3
1.2 What we found, in order 3
2 Background 4
2.1 The task: which nodes are connected? 4
2.2 The model 4
2.3 What theory says 4
2.4 How results are read 4
3 Timeline 6
4 Getting the measurement right 7
4.1 The learning rate confounds width (Q1, Q1b) 7
4.2 Too little data makes wide models memorise (Q1c) 7
4.3 The graphs must not allow shortcuts 7
4.4 Pilots: on honest data almost nothing learns at first 8
4.5 The protocol used from here on 8
5 Width grows almost quadratically with the standard input 10
5.1 First scaling result 10
5.2 The controls 10
5.3 The baseline result 10
5.4 $n = 72$: the data runs out first 11
6 Depth against width 12
7 Reproducing Sanford et al. 2024a 13
7.1 Reproduced, but the data is solvable without connectivity 13
7.2 Their width does not matter, and their encoding is harder to learn 13
8 Input encodings and width 14
8.1 Edge tokens (Sanford's encoding, all-pairs labels) 14
8.2 Precomputed walks: k-hop rows and random-walk features 14
8.3 A training-only hint: auxiliary k-hop loss 14
9 What the model computes inside 15
10 Edges in the attention: edge bias 16
10.1 The idea 16
10.2 Depth 6: almost no width needed up to $n = 56$ 16
10.3 Past $n \approx 72$ the depth-6 model stalls 16
10.4 Depth fixed at 12: the width curve 16
10.5 The number of heads matters at large n 17
10.6 What is still missing: the baseline at the same depth 17
11 What it means 19
11.1 Answers so far 19
11.2 Limits 19
11.3 Next steps 19
12 Appendix: operations 20

1 Summary

1.1 The question

The thesis title is **Analyzing the Effect of Network Width on Graph Transformers** (agreed with the supervisor on 24 September 2026). A transformer's **width** is the length of the vector it keeps for each token. Theory (Sanford et al. 2024a; Yehudai et al. 2025) says that graph connectivity can be **represented** by fairly narrow transformers, but says nothing about whether training finds such a solution. We ask the practical question: **how wide must a transformer be to learn connectivity, and how does that width grow with the graph?** On 13 October the supervisor narrowed the direction: **study how the input encoding changes the width needed.**

1.2 What we found, in order

1. **Measuring width needs care.** A fixed learning rate fakes a width effect, too little data makes wide models memorise, and the standard benchmark graphs can be solved by local shortcuts. We built a graph generator (`swap`) on which every shortcut we could think of fails, and a training protocol under which the measurement is stable (section 4).
2. **With the standard input the width grows like $n^{1.74}$.** Neither more data, more steps, a different learning rate, a fixed-size input nor more depth removes the growth (sections 5–6).
3. **The published experiment does not measure connectivity.** We reproduced Sanford et al.'s experiment (93.0 % / 98.2 % against their 92.9 / 98.0), but a one-line degree rule scores 97.3 % on their data; their input format is also much harder to learn (section 7).
4. **The encoding decides the growth.** Precomputed 2-hop and 4-hop rows lower the exponent to 1.27 and 1.03; standard random-walk features change nothing; an extra training-only loss lowers it to 1.30; edge tokens need 2–3× **more** width (section 8).
5. **Inside the model, connectivity builds up gradually over the layers**, and the answer is read by comparing two nodes' vectors, not stored per node (section 9).
6. **Edges in the attention make width almost free.** With edge bias, 4–7 numbers per node solve every size from 32 to 128 nodes. Getting there required fixing the depth at 12 (6 layers stall past $n \approx 72$ at any width) and using small heads, because the number of heads matters at large n (section 10).

Numbers to remember. Critical width m^* at test pair accuracy 0.95: adjacency rows 22.4 / 33.1 / 44.7 / 59.6 at $n = 32$ / 40 / 48 / 56, growth $n^{1.74}$ (90 % seed bootstrap 1.53–1.97). 2-hop rows $n^{1.27}$, 4-hop rows $n^{1.03}$, auxiliary loss $n^{1.30}$, random-walk features $n^{1.65}$ (same as the baseline). Edge bias (depth 12, head dimension 2): 4.6 / 4.5 / 4.5 / 5.6 / 6.9 / 6.0 / 6.6 at $n = 32$ / 40 / 48 / 56 / 72 / 96 / 128, growth $n^{0.32}$.

2 Background

2.1 The task: which nodes are connected?

A graph is a set of n nodes joined by edges. Two nodes are **connected** if one can walk from one to the other along edges. The model gets a graph and must answer, for every pair of nodes, "connected or not?" – an $n \times n$ table of yes / no answers, the **reachability matrix** R. Example with 6 nodes in two pieces (**components**):

```
0 – 1 – 2      3 – 4 – 5
```

Nodes 0 and 2 are connected (two steps); 0 and 4 are not. R has "yes" for every pair inside {0, 1, 2} and inside {3, 4, 5}, and "no" for every pair across. Connectivity is a **global** property: whether two nodes are connected can depend on an edge far away from both, so no amount of looking at a node's neighbourhood decides it. That is what makes it a good test of whether a model reasons over the whole graph.

2.2 The model

Each node becomes one **token**, a vector of numbers. In the standard encoding (**adjacency rows**) node i's token is row i of $A + I$: a 1 for itself and for each direct neighbour.

node	0	1	2	3	4	5
0	1	1	0	0	0	0
1	1	1	1	0	0	0
2	0	1	1	0	0	0
3	0	0	0	1	1	0
4	0	0	0	1	1	1
5	0	0	0	0	1	1

A linear **read-in** maps each row to a vector of length m, the **width**. Then a stack of transformer **layers** (the **depth** L) updates these vectors. In each layer every node looks at every other node (**attention**), takes a weighted mix of their vectors and adds it to its own, then passes the result through a small two-layer network ($4m$ hidden units). Attention is split into **heads**, each a separate way of choosing whom to look at; a head uses a fixed number of the m numbers (the **head dimension**), so there are $m/$ head-dim heads. At the end, a **bilinear read-out** scores every pair: $\hat{R}_{ij} = h_i^\top W h_j$, symmetrised, with h_i the final vector of node i. The model is trained with a cross-entropy loss on all n^2 entries at once.

Width m is the one number we vary. It is the length of the per-node vector – how much each node can remember about the graph – and it is what theory calls width. Everything else (depth, head dimension, training) is held fixed within a comparison.

2.3 What theory says

- **Sanford et al. 2024a** show that a transformer of depth $O(\log n)$ can represent connectivity with width growing only slowly in the input size (N^ϵ with extra tokens, $\sqrt{N}$ without), using vertex and edge tokens. Their experiment uses width 768 on graphs of at most 19 nodes, so it never tests the narrow-width claim.
- **Yehudai et al. 2025** show that with adjacency rows as input, width linear in n suffices at constant depth.

Both are statements about what can be **represented**. Whether gradient descent finds such a solution, and at what width, is the empirical question here.

2.4 How results are read

- **Pair accuracy:** the fraction of the $n \times n$ answers that are right, on 400 test graphs never seen in training. Because most pairs are connected, answering "yes" everywhere already scores about 0.75

(**always-yes**, 0.745 at $n = 32$ up to 0.77 at $n = 128$). A model stuck at that value has learned nothing.

- **Critical width** m^*: the smallest width at which the final test pair accuracy reaches 0.95. Each seed's curve is interpolated (in $\log m$) between the widths that were run, and the per-seed crossings are averaged. "≤ 8" means the smallest width tried already passed.
- **Growth exponent:** the slope of $\log m^*$ against $\log n$. An exponent of 1 means width grows in proportion to n; 2 means quadratically; 0 means not at all. Uncertainty comes from a **seed bootstrap**: fit the slope for every combination of one seed per n, report the 5th–95th percentile.
- **Fit vs generalise:** the same crossing on **training** graphs (can the model fit?) and on **test** graphs (does it carry over?). When the two differ, the data, not the width, is the limit.

3 Timeline

dates	what happened
24 Sep – 1 Oct	Thesis topic set: the effect of width. Q1: learning rate and data confound width.
2 – 5 Oct	Shortcut audit; new `swap` generator. Q3 pilots, trimmed and fine grids: m^* grows superlinearly.
5 – 7 Oct	Q3 controls: data (128k graphs), steps, learning rate. Q6: fixed-size input. Result: $n^{1.74}$–$n^{1.94}$.
8 – 9 Oct	Q7: depth against width (depth 2 vs 6).
9 – 13 Oct	Reproduction of Sanford et al. 2024a (parts A, B1, B2) on request of the supervisor.
13 Oct	Supervisor: study input encodings. Edge tokens set up.
14 – 20 Oct	Encodings: edge tokens, k-hop rows, random-walk features; more sizes. Probing set up.
21 Oct – 4 Nov	Probing results; third seed; auxiliary k-hop loss; $n = 72$ for the baseline.
3 – 5 Nov	Edge bias: pilot, full sweep, widths below 8, n up to 128. Plateau found past $n \approx 72$.
5 – 8 Nov	Plateau diagnosed as a depth limit; depth 12 adopted; $n = 128$ checks.
8 – 10 Nov	Depth fixed at 12 for the width study; one-depth edge-bias curve completed to $n = 128$.

4 Getting the measurement right

Before width can be measured, three things that look like width effects but are not had to be removed: the learning rate, the amount of data, and shortcuts in the graphs.

4.1 The learning rate confounds width (Q1, Q1b)

First experiment: $n = 24$, depth 2, 2000 training graphs, widths 8–256. With the learning rate fixed at 10^{-3} the accuracy zig-zags with width (0.917, 0.917, 0.781, 0.912, 0.820, 0.596). With the rate tuned per width, the curve is flat at about 0.97 up to $m = 32$. The best rate falls as width grows (10^{-2} at $m \leq 16$, 10^{-3} at $m = 256$), and wide models diverge at high rates without a warm-up.

Decision: every later run uses a 5 % linear warm-up and cosine decay, and the rate is checked per width. In the main protocol a single rate, 10^{-3}, turned out to be within noise of the tuned optimum at every width that matters (section 5.2).

4.2 Too little data makes wide models memorise (Q1c)

Same task with 500, 2000 and 8000 training graphs at equal training steps (Figure 2). With 500 graphs every width memorises: training accuracy is 1.00 and test accuracy is worse than answering always-yes. With 2000 graphs widths 8–64 generalise but 128 and 256 drop to 0.76 / 0.64. With 8000 graphs every width from 16 up reaches 0.99. So width has a **floor** (too narrow cannot fit) and a **data ceiling** (extra width memorises when data is short). Any width measurement must use enough data that the ceiling is out of the way.

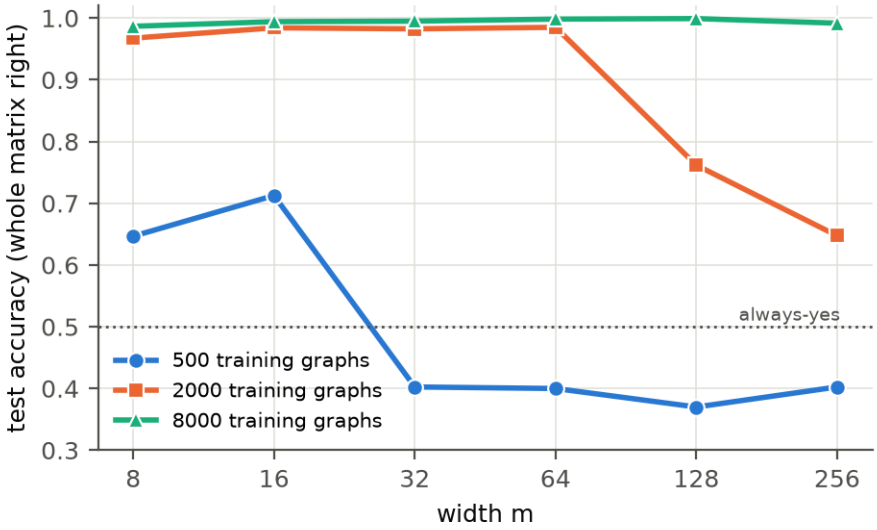


Figure 2: Q1c. Test accuracy (whole matrix right) against width for three training-set sizes, learning rate chosen on validation, mean of 3 seeds. The drop of wide models at 2000 graphs disappears at 8000: it is memorisation, not a property of width.

4.3 The graphs must not allow shortcuts

The first data set leaked the answer. Its two components were always nodes $[0, n_a)$ and $[n_a, n)$, so with adjacency rows the model could read where the split is. Earlier in the year we had also found that the standard "connectedness" data can be solved from node degrees alone. So a shortcut audit (`audit_width_data.py`) was run on every candidate generator: always-yes, k-hop reachability for small k, gradient-boosted trees on graph statistics (edge count, degree histogram, counts of closed walks of length 3–6), and the two combined.

- Graph statistics predicted the label at 0.68–0.90 on small graphs for every existing generator: a bridge between two blobs closes no short cycle, while the matching edge inside a blob does, and closed-walk counts see that.
- On small graphs a 6-hop check is exact on 60–100 % of graphs: the task is local.

The `swap` generator (Figure 3) fixes both. Every graph is two sparse blobs (a cycle plus chords, a quarter of the blob size). In each blob two nodes at least 6 hops apart are picked (a_1, a_2 and b_1, b_2). For label 0 the edges a_1a_2 and b_1b_2 are added (two components); for label 1 the edges a_1b_1 and a_2b_2 are added (one component). Both labels add the same two edges' worth of degree to the same four nodes, and every new cycle has at least 7 edges, so no count of short closed walks differs. Node labels are shuffled randomly.

Audit at $n = 32$ / 48 / 64 / 96 / 128: graph statistics predict the label at 0.53 / 0.49 / 0.51 / 0.50 / 0.51 (chance 0.5); k-hop checks are exact on no graph for $k \leq 4$ (up to $k = 8$ from $n = 96$). The diameter of connected graphs grows like $\log n$ (8.6 at $n = 32$, 12.5 at 64, 16.7 at 128). The generator cannot build graphs below $n \approx 32$.

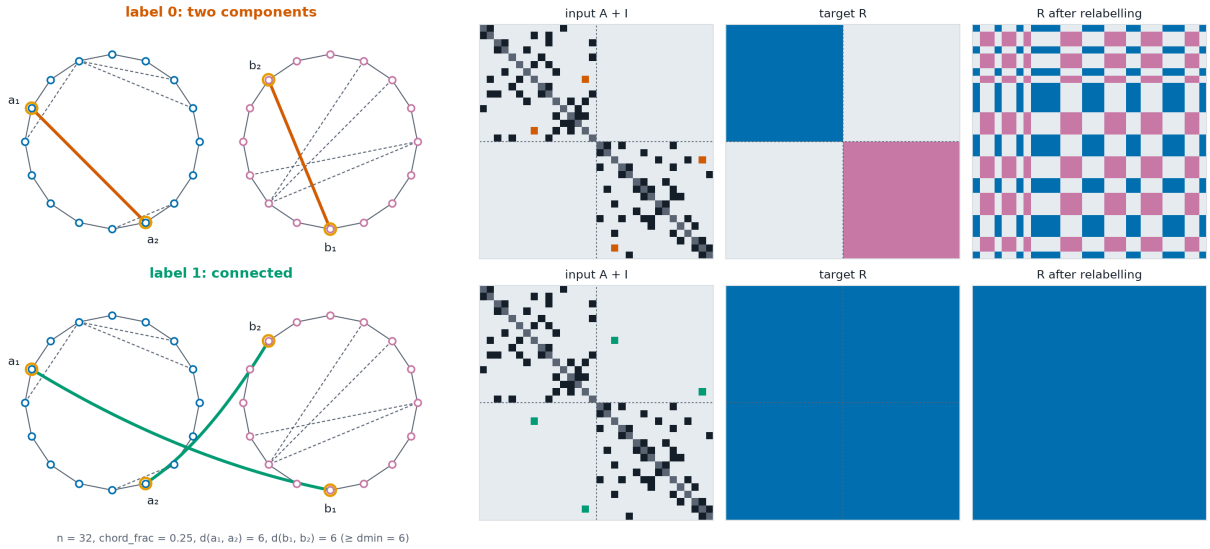


Figure 3: The `swap` generator at $n = 32$. Left: the two labels differ only in how the four marked nodes are joined. Right: the input $A + I$, the target R, and R after the node labels are shuffled (as the model sees it).

4.4 Pilots: on honest data almost nothing learns at first

On `swap` data with the June budget (8000 graphs, 4800 steps), 17 of 18 runs stayed at always-yes, at every width up to 128 and even at the theory's depth $\lceil \log_2 n \rceil$. A **learnability probe** ($n = 32$, 32 000 graphs, 24 000 steps, widths 64–256) then reached 0.998 test accuracy: the task is learnable, the budget was too small, mainly the data. The earlier successes at $n = 16$–24 had relied on locality and statistics.

4.5 The protocol used from here on

choice	value	why
graphs	`swap`, 128 000 train, 400 val, 400 test	enough that fitting and generalising coincide (section 5.2)
steps	24 000, batch 128	doubling to 48 000 changed m^* by nothing (section 5.2)
optimiser	Adam, LR 10^{-3}, 5 % warm-up, cosine	Q1; 10^{-3} is the tuned rate for wide models
regularisation	none	weight decay blocked learning in earlier experiments
depth	6 ($= \lceil \log_2 n \rceil$ for $n = 33$–64)	the depth theory points to; fixed
head dimension	8 (so $m/8$ heads)	only width changes; section 10 revisits this
measurement	final epoch, test pair accuracy 0.95	the "best" checkpoint picked by exact-match was epoch 1 whenever exact-match never moved (a bug found 5 Oct)

choice	value	why
seeds	2–3 per cell, 4 where runs are erratic	seeds agreed within ±3 width units for adjacency rows

This is called the **q3lr protocol** in the code and the changelog.

5 Width grows almost quadratically with the standard input

5.1 First scaling result

On the trimmed grid (32 000 graphs, 12 000 steps) $n = 32$ learned between $m = 32$ and 64, and $n = 64$ and 128 did not learn at any width up to 128: width decides whether learning starts at all. A finer grid ($n = 32$, 40, 48, 56; $m = 16$–128) gave a clean power law: fit $n^{1.53}$, generalise $n^{2.19}$, the gap between them again pointing at data.

5.2 The controls

Each possible artefact was removed in turn:

check	what was changed	effect on the growth
data	32 000 → 64 000 → 128 000 graphs at $n =$ 48 / 56	fit and generalise meet at 128k; growth stays near n^2 ($n^{1.94}$)
steps	24 000 → 48 000 steps at $n = 40$ / 56	m^* unchanged within seed spread
learning rate	$3 \cdot 10^{-3} \to 10^{-3}$ at all sizes	lowers wide models' m^* at $n = 56$; growth $n^{1.74}$
third seed	seed 2 at every bracketing width	exponent unchanged (1.74), interval 1.53–1.97
input size	replace the n-wide row by a fixed 96-wide random code (Q6)	growth unchanged ($n^{2.03}$), 20–25 % more width

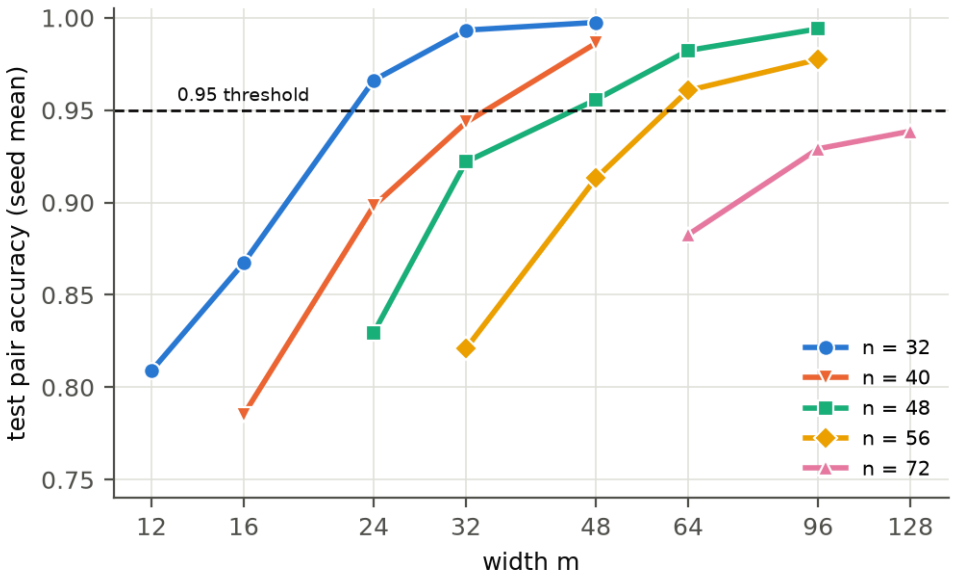


Figure 4: Adjacency rows, depth 6, q3lr protocol: test pair accuracy against width, mean over seeds. Each larger graph needs a wider model to cross 0.95; $n = 72$ does not cross at all with 128 000 graphs (section 5.4).

5.3 The baseline result

	$n = 32$	40	48	56	growth
m^* (3 seeds)	22.4	33.1	44.7	59.6	$n^{1.74}$ (1.53–1.97)
per seed	20.7 / 23.1 / 23.4	30.8 / 29.4 / 39.1	41.2 / 43.6 / 49.4	59.2 / 56.7 / 63.1	

	$n = 32$	40	48	56	growth
m^*/n	0.70	0.83	0.93	1.06	

The width to **learn** connectivity grows clearly faster than linearly, far above the slowly growing width that suffices to **represent** it (Sanford et al.) and above Yehudai et al.'s linear construction.

5.4 $n = 72$: the data runs out first

At $n = 72$ (widths 64–128) the **training** fit crosses 0.95 at $m^* = 81$, exactly where the $n = 32$–56 trend puts it (82.6). But no run reaches 0.95 on test graphs, even at $m = 128$ or with doubled steps: the training graphs no longer cover enough pairs. Doubling the training set to 256 000 brings the train–test gap from 0.04–0.08 down to 0.02 and lets one seed pass at $m = 96$. The baseline beyond $n = 56$ would need much more data and compute, which is one reason the encodings below were tested mostly at $n \leq 56$.

6 Depth against width

Depth is not the variable of this thesis, but it had to be checked once whether the width growth is just a depth shortage in disguise (Q7, 8–9 October).

- **At $n = 40$, more layers save only a third of the width** (Figure 5 a): m^* is 45.3 at depth 2, 30.1 at depth 6 and 32.3 at depth 8. Two layers already learn, on graphs of diameter about 10, so the model does not trace paths one hop per layer.
- **At depth 2 the width grows faster, not slower** (Figure 5 b): m^* 28.2 / 45.3 / 92.7 / > 192 at $n = 32$ / 40 / 48 / 56, about n^3. Yehudai et al.'s linear-width, constant-depth construction is not what training finds.

So depth and width trade only mildly, and the growth of m^* at fixed depth is real. From here on depth is a fixed setting and only width is varied.

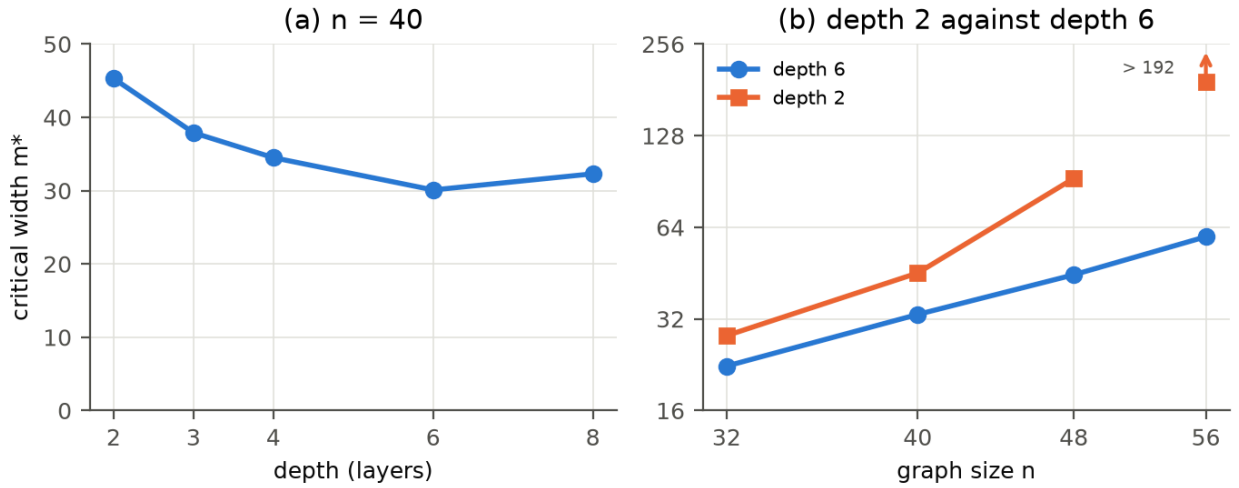


Figure 5: Adjacency rows. (a) Critical width against depth at $n = 40$. (b) Critical width against n at depth 2 and depth 6 (log–log).

7 Reproducing Sanford et al. 2024a

At the supervisor's request we checked whether the experiment behind the theory reproduces. Sanford et al. train a 12-layer, width-768 decoder on GraphQA "reachability" (random graphs with 5–19 nodes, one source–target question per graph), with one token per vertex, one per edge and task tokens, and report 92.9 % (1000 training graphs) and 98.0 % (100 000).

7.1 Reproduced, but the data is solvable without connectivity

	theirs	ours
1K training graphs	92.9	93.0
100K training graphs	98.0	98.2 (needed LR 10^{-4}; their $5 \cdot 10^{-4}$ did not train)

On the same test distribution:

rule or model	test	hard: 4+ hops	hard: no path
always yes	0.833	–	–
"yes iff both endpoints have an edge"	0.973	1.000	0.000
within 3 hops	0.981	0.000	1.000
boosted trees on 2-hop neighbourhoods	**0.995**	0.932	0.893
reproduced transformer (width 768)	0.982	0.81–0.89	0.63–0.70

A one-line degree rule is as good as their transformer, and a 2-hop learner is better. On graphs of at most 19 nodes the 2-hop neighbourhood covers most of the component, so the data cannot separate computing connectivity from local statistics. The "hard" columns are a test set we added (connected pairs at least 4 hops apart; disconnected pairs whose endpoints both have edges): the transformer is below the 2-hop learner there at every width.

7.2 Their width does not matter, and their encoding is harder to learn

B1: on their data, width 32 already gives 98.3 %; widths 8 to 768 all land at 0.957–0.984. GraphQA saturates on local shortcuts at $m \approx 32$, so it cannot measure a critical width.

B2: their encoding (vertex + edge + 32 query tokens) on our `swap` graphs, same budget as the baseline: every run at every width up to 96 stays at always-yes, even at 100 000 steps (runs that left the plateau memorised). Section 8.1 shows this is the encoding, not the 32× smaller number of labels.

8 Input encodings and width

From 13 October the question is: **how does the input encoding change the width needed?** Every encoding below is tested under the same protocol as the baseline (depth 6, head dimension 8, 128 000 graphs).

8.1 Edge tokens (Sanford's encoding, all-pairs labels)

One token per vertex (learned ID, no positions) and one per edge, with the same all-pairs read-out as the baseline. At the baseline budget no size reaches 0.95 (best: $n = 32$, $m = 48$, 0.921 pair accuracy). With 4× the steps a critical width exists: 48.2 at $n = 32$ (2.2× the baseline) and at least 72 at $n = 40$ ($\geq$ 2.4×). Runs plateau for long stretches and seeds disagree. **Conclusion:** with the same labels, edge tokens are harder to learn than adjacency rows; Sanford's representation bound for this encoding ($\sqrt{N} \approx 9$–10) is far below the width that learns.

8.2 Precomputed walks: k-hop rows and random-walk features

Two kinds of extra input per node, appended to the adjacency row:

- ***k*-hop rows:** the row of $[(A + I)^k > 0]$, i.e. which nodes are within k steps. This hands the model k hops for free; what remains is connectivity over about diameter $/k$ hops.
- **RWSE:** the probability that a random walk returns to the node after $j = 1...K$ steps (Dwivedi et al. 2022), a standard structural encoding in graph transformers.

A leak audit came first: with $K = 16$ (the GraphGPS default) RWSE predicts connectivity at 0.90 (graph level, $n = 40$) on its own, because long walks approach a value set by the component size. $K \leq 8$ is clean (0.50–0.54) and was used.

input	$n = 32$	40	48	56	72	growth (90 % interval)
adjacency rows	22.4	33.1	44.7	59.6	–	$n^{1.74}$ (1.53–1.97)
+ RWSE, $j \leq 8$	23.3	31.9	45.2	57.9	–	$n^{1.65}$ (1.46–1.84)
+ 2-hop rows	9.5	12.7	16.9	18.5	27.1	$n^{1.27}$ (1.10–1.51)
+ 4-hop rows	3.5	3.6	3.7	4.5	6.5	$n^{1.03}$ (0.79–1.25)

- **Every free hop lowers both the level and the growth.** Two free hops cut the width 2.4–3.2× and the exponent from 1.74 to 1.27; four free hops (about 3 of the 10–13 hops left) bring the width to 4–7.
- **RWSE changes nothing**: same width as plain rows at every size.
- **A leak lowers the measured width**: the leaky RWSE-16 needed 17–22 % less width, less so as the leak fades with n. This is why every encoding was audited before it was trusted.

8.3 A training-only hint: auxiliary k-hop loss

Can the width fall without changing the input? During training, layers 1, 2 and 3 of the baseline model also predict 2-, 4- and 8-hop reachability through extra read-out heads (weight 0.3 each); the heads are dropped at test time, so the tested model is the baseline model.

	$n = 32$	40	48	56	growth
adjacency rows (seeds 0 / 1)	21.9	30.1	42.4	58.0	$n^{1.74}$
+ auxiliary loss	21.5	26.8	33.5	45.1	$n^{1.30}$ (1.13–1.45)
saving	2 %	11 %	21 %	22 %	

The saving grows with n, so the hint changes the exponent, not only the constant. It reaches about the 2-hop rows' exponent, at a higher level: organising the computation slows the growth, extra information in the input mainly lowers the level.

9 What the model computes inside

Linear **probes** were trained on the frozen baseline models at $n = 40$ and 56, at widths below and above m^*, on fresh graphs: from each layer's node vectors, can a simple bilinear form ($h_i^\top W h_j$, the model's own read-out form) tell whether i and j are within k hops, or connected at all? Every probe is also fitted on the same network **untrained**, as a baseline for what the probe can do by itself.

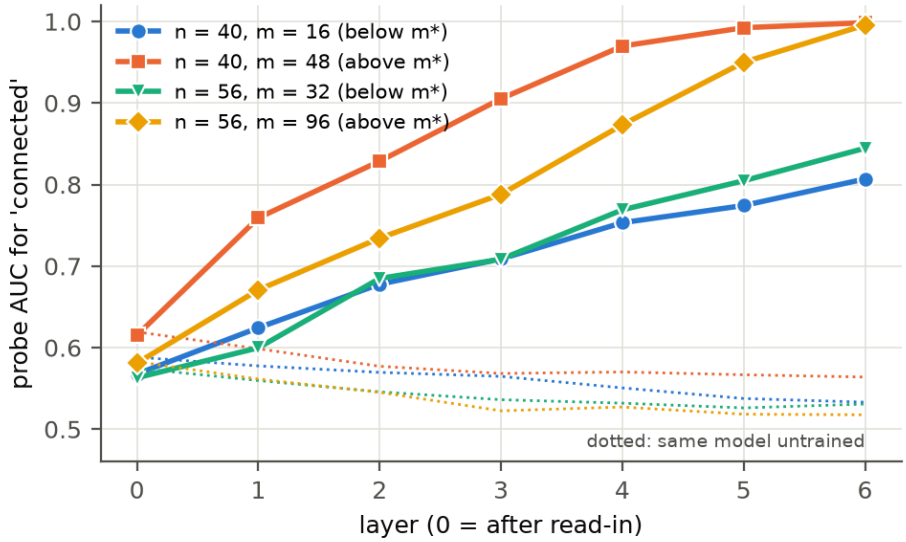


Figure 6: Probe for "connected" (ROC-AUC) after each layer, mean of 2 seeds. Solid: trained models; dotted: the same networks untrained.

- **Connectivity builds up gradually**, a little in every layer, with no single step where it appears (Figure 6). Models above m^* reach 0.99–1.00 at the last layer; narrow models rise more slowly and end at 0.81–0.84.
- **There is no hop-by-hop hierarchy.** The 3-hop signal appears in layer 1 and stays flat; exact 1-hop information fades in late layers. The model builds a "same component" signature rather than doubling its reach per layer.
- **Nodes do not store their reachable set.** From a single node's vector its connected set is decodable only at AUC 0.60–0.89; the answer comes from comparing two nodes' vectors.

10 Edges in the attention: edge bias

10.1 The idea

In every attention layer, add a learned bonus $b_{\ell,h} \cdot (A + I)_{ij}$ to the attention score between i and j: one number per layer and head, starting at 2. Each head can then choose to look at neighbours (large b) or at everyone (small b). The tokens stay adjacency rows. This is the edge part of Graphormer-style attention biases; unlike shortest-path biases it gives no global information, only "who is my neighbour".

A first pilot showed why the start value matters: with b starting at 0 the model sees no structure at first and learns slowly; starting at 2, $n = 32$ is solved at $m = 16$ within a fraction of the baseline budget.

10.2 Depth 6: almost no width needed up to $n = 56$

Full q3lr protocol, depth 6. With head dimension 8 every width from 8 passed at $n \leq 48$, so widths below 8 were tested with head dimension 2 (so that $m = 4$ still has 2 heads): $m^* = 5.6$ / 5.7 / 4.1 / 5.9 at $n = 32$ / 40 / 48 / 56, against 22–60 for the baseline. The input carries exactly the same information as the baseline's; only the attention knows where the edges are.

10.3 Past $n \approx 72$ the depth-6 model stalls

At $n = 72$, 96 and 128 (still depth 6) m^* rose again: 13–15, 16–28, and above 22. Runs sat on a plateau at 85–93 % pair accuracy and either escaped or did not, almost at random; accuracy was not even monotone in width. Three explanations were tested:

- **The bias gets diluted** as n grows (a fixed bonus puts less attention on neighbours when there are more non-neighbours). True in the diagnostics (31–49 % of attention on neighbours at $n = 128$ against 55–63 % at $n = 72$), but fixes for it (bias growing with $\log n$, scaled softmax) did not help.
- **The learning-rate schedule runs out.** Gradient clipping and a warmup–stable–decay schedule helped a little.
- **Six layers are too few at this size.** A stalled model, run through its own 6 layers twice (no new parameters), solved the task within 6000 steps (0.994–1.000) in every case; 12-layer models trained from scratch did not stall.

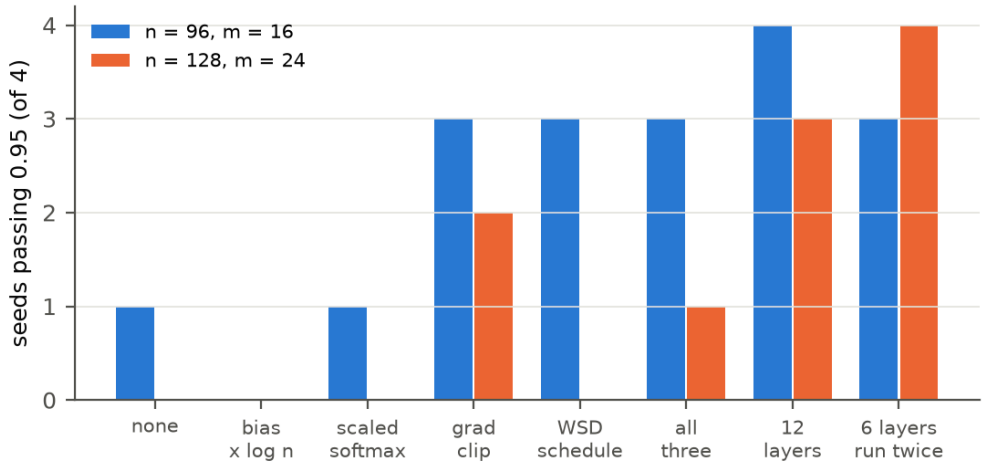


Figure 7: Plateau ablation with edge bias at depth 6: one change at a time, 4 fresh seeds per condition. Bars count seeds reaching 0.95. Only more layers, or the same layers run twice, remove the stall.

So at depth 6 the rise of m^* past $n \approx 72$ measured a depth shortage, not a width requirement. For the width study this means: the depth has to be fixed **once**, high enough that it is not the limit for any n in the study, and then left alone. We fixed it at **12 layers**.

10.4 Depth fixed at 12: the width curve

With depth 12 and head dimension 2, all sizes were rerun (Figure 8):

n	32	40	48	56	72	96	128
m^* (seed mean)	4.6	4.5	4.5	5.6	6.9	6.0	6.6
seeds	2	2	2	2	2	2	4

The growth exponent over $n = 32$–128 is **0.32**: the width needed barely changes while n grows fourfold. Runs are all-or-nothing: a run either solves the task (≥ 0.98) or stays at always-yes, with little in between (Figure 8). At $n = 128$ one seed passes at $m = 6$, fails at $m = 8$ and passes again at 12, so single runs are noisy; with 4 seeds, 3 pass at $m = 6$ and all 4 at $m \geq 12$.

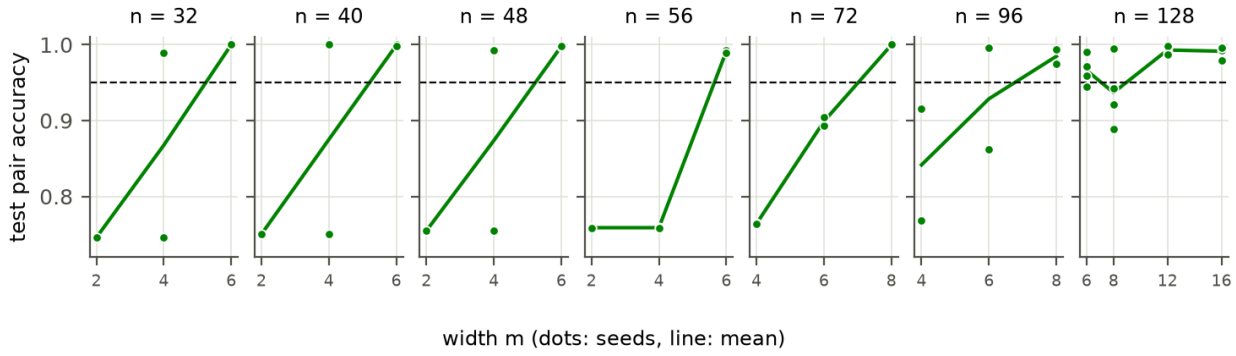


Figure 8: Edge bias at depth 12, head dimension 2: test pair accuracy against width for every n. Dots are seeds, the line their mean; dashed: the 0.95 threshold.

10.5 The number of heads matters at large n

The first depth-12 runs used head dimension 8 (1–2 heads at $m = 8$–16). At $n = 72$ and 96 they gave about the same m^* as head dimension 2 (≤ 8), but at $n = 128$ they needed about 14.5, and 16 layers did not fix it (11 by median, with late collapses). With head dimension 2 (3–8 heads) the same $n = 128$ gives $m^* \approx 6.6$ and no collapses (Figure 9).

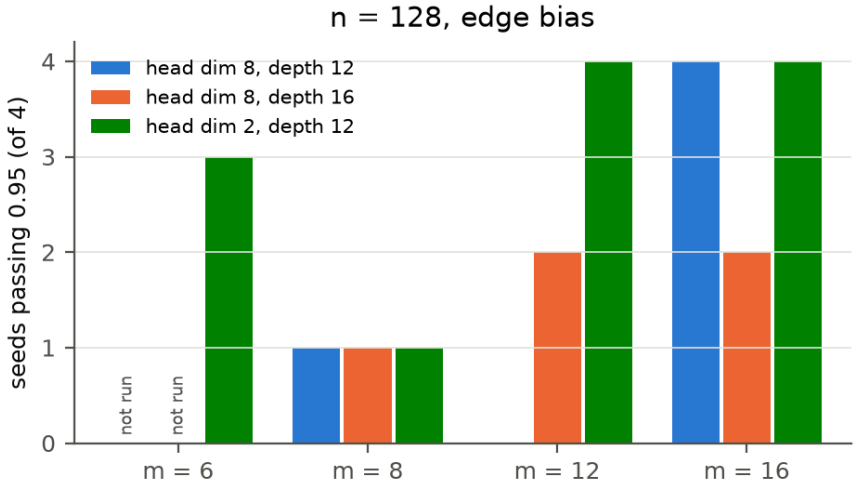


Figure 9: $n = 128$, edge bias: seeds (of 4) reaching 0.95 at each width, for head dimension 8 at depth 12 and 16, and head dimension 2 at depth 12.

So the head dimension is not neutral: with few heads, each head must serve several purposes (neighbour-only and broad attention), and at large n that is the bottleneck. The thesis fixes head dimension 2 for the edge-bias curve and reports this comparison as the reason.

10.6 What is still missing: the baseline at the same depth

The edge-bias curve is at depth 12; the baseline curve is at depth 6. To claim that edge bias lowers the width **at equal depth**, the baseline must be measured at depth 12 too. A first attempt used head dimension 2 for the baseline as well; it failed at every width (no run reached 0.95; e.g. $n = 32$, $m =$

24: 0.911 against 0.96–0.98 at depth 6 with head dimension 8). Without edge bias, attention must find neighbours by comparing query and key vectors, and 2-number heads are too small for that; with edge bias the bias does the finding. The baseline therefore needs its own head dimension (8) at depth 12. That run (adjacency rows, depth 12, head dimension 8, $n = 32$–56, about 2–3 hours on Kaggle) is the next experiment.

11 What it means

11.1 Answers so far

1. **With the standard encoding, the width to learn connectivity grows like $n^{1.74}$**, robustly across data, steps, learning rate, seeds and input size. This is far above what theory says is needed to represent connectivity: training does not find the narrow solutions.
2. **The encoding sets the growth.** In order of width needed: edge tokens (more than the baseline) $>$ adjacency rows $\approx$ RWSE $>$ auxiliary loss $>$ 2-hop rows $>$ 4-hop rows $\approx$ edge bias. The exponent falls from 1.74 to about 1 with precomputed hops, and to about 0.3 with edge bias.
3. **Edge bias is the cheapest change that removes the growth.** It adds one number per layer and head, gives the model no global information, and leaves the input unchanged, yet 4–7 numbers per node suffice from 32 to 128 nodes.
4. **Two settings must be fixed and stated, not varied:** the depth (12; 6 is too shallow for edge bias at $n \geq 72$) and the head dimension (2 for edge bias; 8 for adjacency rows, which need larger heads to find neighbours by content).
5. **Benchmarks need shortcut audits.** GraphQA reachability, the standard connectedness data and RWSE with long walks all leak the answer; a leak shows up as a smaller measured width.

11.2 Limits

- Two seeds per cell for most points; edge-bias runs are all-or-nothing, so single crossings are noisy. The $n = 128$ point has 4 seeds.
- Graph sizes 32–128; the baseline only to 56 (72 is data-limited).
- One graph family (`swap`), one task (all-pairs connectivity).
- The comparison across encodings mixes depth 6 (all except edge bias) and depth 12 (edge bias) until the depth-12 baseline is in.

11.3 Next steps

1. Baseline (adjacency rows) at depth 12, head dimension 8, $n = 32$–56 (Kaggle, about 2–3 h).
2. More seeds for the edge-bias curve where runs disagree ($n = 96$, 128).
3. Write the results into thesis chapters 3–4; update the bootstrap interval for the baseline (1.53–1.97, from three seeds) where the two-seed interval (1.59–1.90) is still quoted.

12 Appendix: operations

- **Compute.** Sweeps run on Kaggle (2× T4, 12 h sessions, notebook `kaggle/width_sweep.ipynb`) and, since 3 November, on Colab Pro through its command-line tool, with a local watcher that pulls results and stops the VM. Colab balance went from 91.45 to 17.41 units.
- **Incidents that cost runs.** A 2-hour lock timeout killed six shards (6 Oct); 64 parallel workers ran out of memory (19 Oct); a worker killed while building $n = 128$ data hung the notebook (7 Nov); shards that started late took a shifted slice of the job list, running 3 runs twice and skipping 6 (7 Nov); Colab VMs died when the laptop lost network (3, 5 and 9 Nov). All were fixed in the code; results were recovered or rerun.
- **Reproducibility.** Every run writes one JSON line (configuration, learning curve, final metrics) to `results/width/<sweep>*.jsonl`; `width_sweep.py <sweep> --scaling` recomputes every critical width in this report; `docs/CHANGELOG.md` records each day's runs, numbers and decisions. GPU training is not bit-for-bit deterministic: the same run repeated gave 0.995 and 0.938 once.